

Prompt方法论·三项目实战版

用途：跨项目可复用的Prompt工程方法论·附三个项目真实提示词片段与优化前后对比

项目：小办（企业RAG办事问答）·经历翻译官（AI求职诊断）·缓择星球（AI多智能体决策助手）

v2升级：方法论模板重构为「角色设定->任务拆解->约束->Few-shot ->输出Schema」；新增System Prompt / CoT / 上下文窗口与记忆压缩三个专项；覆盖三项目真实源码（小办config/*.json、经历翻译官 backend/prompts.py、缓择core/agents/prompts/*.md）

0.我的主张（一句话）

Prompt不是"把需求说清楚"，而是把模型的自由度收敛到「恰好够用」的范围：

该钉死的地方钉死（角色/流程顺序/输出格式/禁止项/兜底边界），该放开的地方放开（内容组织、表达风格）。

三个项目横跨三类任务，正好验证了同一条方法论：

- 小办是"事实型问答"——要准，约束往死里钉（否则制度答错）；
- 经历翻译官是"生成型诊断"——要个性化，钉死结构、放开表达（否则全是模板废话）；
- 缓择星球是"对话型决策辅助"——要中立不讨好，钉死协议、控住情绪（否则变成附和机器）。

踩过的坑几乎都出在同一处：约束写得太软，模型就自己发挥；约束写得太死，输出就模板化。这份方法论就是在这两者之间找边界的经验。

1.方法论模板（可直接复用）：角色设定->任务拆解->约束->Few-shot ->输出Schema

① 角色设定	—— 限定身份、唯一 KPI、说话基调，收窄"想做什么"的空间
② 任务拆解	—— 用有序步骤写死执行顺序与判断口径，禁止跳步、禁止自由发挥
③ 约束	—— 正向要求 + 负面禁止 + 分层兜底 + 克制边界
④ Few-shot	—— 给真实正反例，教它"边界"而不是只教它"模仿"
⑤ 输出 Schema	—— 定死结构与判定字段，让下游代码 / 评测可直接消费

模板落地对照表（三项目x五段）

段	小办normal_answer	经历翻译官report	缓择agent3魔鬼代言人
①角色设定	你是「小办」答案生成节点，唯一任务是根据上游检索结果结构化作答	你是「经历翻译官」跨行业能力迁移顾问，产出个性化诊断报告	你是「决策魔鬼代言人」，唯一KPI是帮用户避免未来后悔，哪怕要唱反调；"附和=失职，沉默=帮凶"
②任务拆解	步骤1标准QA优先->步骤2解析片段(有效正文/仅标题/无关)->步骤3分支判定->步骤4组装JSON；严格按顺序、不可逆	六大模块顺序固定(parse->skills->match->resume->interview->tips)，内部组织自由	【强制反向协议】1 Steel-man先共情->2找矛盾->3亮明反向（必须指具体选项/硬伤）->4自我核验（不达标必须重写）
③约束	严禁相似度阈值兜底；仅标题/无关才兜底；分层话术	去模板化硬约束；严禁编造；置信度透明	克制原则(一句话点透、不推销、不种草词)；reverse_intensity强弱分级；轮数上限自动降级；中性表达(性别刻板印象禁止)
④ Few-shot	【别报】真实违规case -> 【实测】正确作答对照	5种风格画像九维注入（当作"风格few-shot"）	agent3_few_shots.json 4条真实对话：用户表态->完整反向JSON（含征询式引用）

段	小办normal_answer	经历翻译官report	缓择agent3魔鬼代言人
⑤输出Schema	纯JSON: answer_text/source/handoff_label/handoff_url	纯JSON: 6键x每键1-4张{title,content}卡片	纯JSON: challenged_preference/recommended_alternative/reverse_intensity/contradiction/message

每段的通用动作（照着写就够用）

- ①角色设定（3个动作）
1. 给一个一句话身份+一句话唯一KPI（KPI要可判断：缓择的"避免后悔"比"当助手"有用一百倍）；

2. 说明"你不干什么"（小办：你不是客服闲聊；缓择：你不是推销员）；

3. 给一个情绪基调（缓择："直接但友善的朋友"；经历翻译官："温和求职陪练，不贩卖焦虑"）。
- ②任务拆解（2个动作）
1. 把"要不要X"这类模糊判断改写成可枚举的分支步骤（步骤N判定什么、每步跳转去哪）；

2. 在该停的地方强制自检（缓择agent3第4步自我核验、小办normal_answer步骤3分支判定）——这是CoT的可控用法，见§3.2。
- ③约束（4个动作）
1. 显式负面约束："禁止X /严禁Y"，只写正面要求约等于没写；

2. 动态约束：根据用户最新表态切换禁止项（缓择预算分流：用户说省钱->禁止从价格切入、禁止推荐更贵选项，见案例E）——约束不是死的，是跟着上下文走的；

3. 分层兜底：不同"不知道"给不同话术+给人工出口；

4. 克制边界：字数上限、一次只问一件事、不无限追问、不替用户做最终决定。
- ④ Few-shot（3个动作）
1. 优先放反例（[别报] -> [实测] 对照），反例教边界、正例教模仿；

2. few-shot必须来自真实回归badcase，不是手写的理想例子；

3. 复杂协议用"用户话->完整JSON输出"成对示例（缓择few_shots每条都是一个完整的输入输出对，模型照着学协议结构）。
- ⑤输出Schema（3个动作）
1. 字段给语义注释（reverse_intensity的strong/weak各自什么含义）；

2. 加判定字段让"做没做到"可程序判定（小办handoff_label、缓择challenged_preference +contradiction）——这是把定性能力变成可回归验收项的关键；

3. 写死"只输出JSON，无额外文字/无markdown代码块/无解释"。

2.三个项目的提示词资产地图（素材来源，全部真实）

项目	资产位置	清单	分工方式
小办	xiaoban-workflow-src/config/*.json	6份：query_rewrite / split_and_classify / normal_answer / fallback_answer / biz_classify / question_split	每份JSON内含sp(System)与up(User)两段，sp/up物理分离
经历翻译官	backend/prompts.py	报告主指令+6类局部改写指令+5种风格画像(STYLE_PROFILE) +6类咨询对话问题库	Python常量组织，风格画像9维代码块注入
缓择星球	core/agents/prompts/*.md +core/nodes/*.py	agent1~4_system.md（四Agent）+agent_rehearsal / agent_clarify / agent_reflection / agent3_few_shots.json +场景指南(scenario_guides)	System Prompt独立成.md文件+few-shots独立.json；节点代码负责拼User上下文

组织启示：三项目不约而同都做了System与User/上下文分离——这既是工程整洁，也是缓存与维护的刚需（见§3.1）。

3.三个专项技法（System Prompt / CoT / 上下文窗口与记忆压缩）

3.1 System Prompt设计：什么进System，什么绝不能进

判断标准一句话：System放"这次会话永远不变的规则"，User放"每次都在变的输入"。

四个通用动作

- 1. 角色+KPI +基调放System（见模板①）；
- 2. 规则编号化：System里每条约束给编号（"规则3：意图保真（最高优先级）"），User侧Prompt可以直接引用编号（"仅按规则1-6执行"），降低歧义；
- 3. 输出Schema放System，并给字段注释——Schema是全局契约，不该每次重复；
- 4. 别把大段示例塞System头部：示例属于Few-shot，放末尾；若模型对中段注意力差，把最重要的约束放System开头和结尾（首因/近因效应真实存在）。

三项目实证

- 小办：sp全部是固定规则（角色、强制规则、兜底话术、Schema），up才是当次的问句+检索片段->这同时服务前缀缓存（sp固定即可命中，检索动态内容放user尾部，见成本账）。
 - 缓译：System独立成agent1_system.md等文件（角色锁定、强制协议、Schema、克制原则全在System），User侧由节点代码拼JSON（场景/需求/选项/画像/记忆块）——System像宪法、User像当庭证据。
 - 经历翻译官：风格画像整体作为System级注入，报告内容随User变化。
- 坑：把"用户的具体经历/问句"写进System ->前缀缓存失效+每条链路token虚高；把"规则"写进User ->每次重复消耗+规则容易被长上下文稀释。

3.2 CoT（思维链）：三项目里CoT长什么样、怎么防失控

我的立场：不开放"自由推理"（会绕、会编），而是把推理写成显式的、可检查的步骤协议——相当于给思维链套上轨道。

项目	CoT形态	具体写法
小办	分支判定链	normal_answer"步骤1-4严格按顺序、不可逆"；split_and_classify要求多意图逐条拆分、逐条分类，严禁一条答不出就整题兜底
缓译	反向推理链	agent3【强制反向协议】4步：Steel-man共情->找矛盾（"找不到就重读需求与画像，矛盾一定存在"）->亮明反向->自我核验（"如果答案是'否'，输出不合格，必须重写"）
经历翻译官	结构化输出链	六大模块按固定顺序生成，后模块引用前模块结论（如"岗位匹配度"必须引用"可迁移能力"的评级）

CoT三个防失控动作（生产可用关键）

- 1. 每步只给一个判定点：步骤= "输入什么->看什么字段->输出什么结论"；
 - 2. 强制自检步骤（缓译第4步"自我核验"是我见过性价比最高的防绕手段——它把"模型以为自己答了"变成"必须确认自己答了"）；
 - 3. 给CoT设出口：答不出来/矛盾消解不了->走兜底或降级，不许绕来自圆其说（缓译reverse_intensity=weak降级态、小办场景A/B兜底话术）。
- 要点：我不让模型"think step by step"自由发挥，而是把推理写进System变成"强制协议"——CoT的价值不在"想"，在"每一步都可检查"。

3.3上下文窗口与记忆压缩：三项目各一种做法

核心矛盾：多轮产品里上下文会越攒越长——撑爆窗口、稀释注意力、抬高成本。解法不是"全塞进去"，是压缩+分层+征询。

项目	做法	本质
缓择	征询式记忆引用（core/memory/consultative.py）：把冗长历史压缩成画像块——历史后悔模式、决策风格、场景记忆、已学偏好倾向，每行一行摘要；并约束使用句式："引用历史时必须用『你上次说.....这次还是吗?』，不冷冰冰断言"	记忆压缩成画像+把"怎么用记忆"也写进约束（隐私友好+防断言错误）
小办	检索上下文裁剪：up只带当次问句+命中的QA/片段（不整段塞制度文档）；多轮追问时携带上一轮结论摘要而非全文	上下文只放"这次决定需要的"
经历翻译官	双模式（深度/极速）：极速模式主动压缩——精简面试问答与成长建议、保留核心结论；对话侧按6类问题库裁剪上下文范围	按使用场景主动降上下文预算

通用动作

- 1. 先算窗口预算：System(固定) + 记忆(压缩后) + 检索/输入(当次) + 输出预留——四项加起来超了要砍，砍的顺序是"历史细节>示例>规则冗余"；
- 2. 历史摘要化：不是把旧消息全量回放，而是让一个节点把旧会话压缩成"画像/结论块"再注入（缓择 profile_store / learning）；
- 3. 引用历史要"征询"：隐私产品尤其——先问"是否参考你上次说的XX"，而不是默认用户记得自己说过；
- 4. 注意token成本：压缩掉的每一段都是省下的钱（详见《成本账》：前缀缓存要求sp固定、动态内容置后）。

4.实战模式库（三项目全量，共11个模式）

模式1|把"判断"写成"有序流程"，禁止跳步

- 坑：让模型自由判断"这个能不能答"，它优先选最省事的兜底。
- 写法（小办normal_answer）：

```
# 执行流程（严格按顺序执行，不可逆）
步骤 1：标准 QA 优先判定 —— standard_hit=true 直接引用预制答案，跳到步骤 4
步骤 2：解析 knowledge_retrieval，把片段分为【有效正文】/【仅标题】/【无关或空白】
步骤 3：回答分支判定（核心）—— 有有效正文必须正常回答；仅标题走场景 A 兜底；无关走场景 B 兜底
步骤 4：组装并输出 JSON
```

- 效果：把"要不要兜底"从模糊判断变成可枚举、可回归的三支决策。

模式2|显式负面约束+错误示例对照（我最贵的一条经验）

- 坑：只写"请根据检索结果回答"，模型用相似度分数当兜底依据——库里有答案，却因"分不够高"输出兜底。
- 写法（小办）：

最高优先级强制规则

1. 【严禁相似度阈值兜底】无论相关度 0.3 还是 0.9，只要片段内存在【有效正文】（金额、天数、审批、流程、时间、比例等具体规则），就必须正常作答，禁止以“分不够高”为由兜底。
6. 【兜底话术限制】只有在步骤 3 判定为“仅标题”或“无关/空白”时才允许兜底。存在有效正文时输出兜底，判定为严重违规。

[别报] 错误示例（真实违规 case）

输入：相关度 0.70，正文“所有报销申请应在支出发生后 1 个月内提交”

输出：“该问题在公司现行制度中未明确规定...” <- 严重违规

[实测] 正确示例

-> 基于“1 个月内提交”这条有效正文，正常输出政策结论 + 路径 + 步骤 + 来源

- 效果：配合检索侧改造（看“有没有业务正文”而非向量分），118条实测整体91.5%、核心业务零误答。

模式3|意图保真：给“不要做什么”设最高优先级

- 坑：改写节点为提升匹配率，偷偷“帮用户补全条件”（“报销”->“出差报销”），替用户做假设。
- 写法（小办query_rewrite）：

规则 3：意图保真（最高优先级）

- 不能主动增加用户没提到的条件（用户没说金额，不要补“500 元”）
- 不能缩小或扩大用户询问范围（用户说“报销”，不要擅自改为“出差报销”）
- 禁止直接回答问题，你只改写问句，不做解答

- 效果：改写只做“语言规范化+术语对齐”，不做语义推断。

模式4|分层兜底：不同的“不知道”给不同的话

- 写法（小办）：

场景 A（仅检索到章节标题，无匹配细则）：

“已检索到【XX 制度】相关条目，但暂未查询到与您问题匹配的具体规定，建议请示直属上级或联系 HR。”

场景 B（无任何相关文档）：

“该问题在公司现行制度中未明确规定，建议请示直属上级或联系 HR。”

- 效果：场景A至少告诉用户“制度存在只是没查到”，降低“系统坏了”的感知，且给人工出口、不硬编。

模式5|风格注入：换风格=换句子构造，不是换形容词

- 坑：说“用温暖的语气写”，模型只会塞形容词。
- 写法（经历翻译官STYLE_PROFILE，9维代码块）：

```
STYLE_PROFILE = {
  "逻辑严谨型": {
    "voice_one_liner": "像冷静的数据分析师，先给结论再给证据",
    "tone": ["专业、克制", "中性、零情感词"],
    "sentence": "短句为主 (<=25 字/句)；动词前置；量化表达",
    "format": ["结论用加粗前置", "大量表格与编号列表", "结尾给风险/边界"],
    "opening": "首句必须是结论或数据，禁止寒暄",
    "length": "段落 <=4 行，超过即拆表格",
    "vocabulary": ["置信度", "阈值", "归因", "反例"],
    "exclusive": "允许数学符号与中英混排（首次给中文释义）",
    "forbidden": ["[别报] 我觉得/或许/可能吧", "[别报] 感叹号表情", "[别报] 故事比喻"],
  }, ... # 共 5 种
}
```

- 收尾钉一句：“换风格不是换形容词，是换整篇的句子构造、句长分布、表格/列表比例与开篇位置。”

- 效果：5种风格结构差异肉眼可辨（共情叙事型禁表格、务实落地型强制Checklist + 明天第一件事）。

模式6|去模板化硬约束+置信度透明

- 坑：模型最擅长"放之四海皆准"的漂亮废话。
- 写法（经历翻译官）：

【去模板化硬约束（最重要）】

- 严禁产出通用话术，每条结论必须锚定这份具体经历/目标岗位
- 每个模块 title 必须基于内容定制（如「从 0 到 1 搭起社群增长闭环」），禁止通用标题
- 严禁编造不存在的经历与成果；信息不足处标注「待补充」

置信度判定要透明可解释：

高 = 完整闭环 + 可量化成果 + 核心动作

中 = 有经历但缺岗位视角与数据

低 = 仅基础工具/理论无实战

- 效果：输出"一看就是为这个人写的"，用户也知道哪条更值得信。

模式7|输出Schema收敛：让下游代码和评测能直接消费

- 写法：三项目全部纯JSON + 字段注释+判定字段+ "只输出JSON无额外文字"。
- 效果：节点间不用文本清洗；"结构合规"成为可回归的验收项；缓择让"反了没有"可程序判定（见模式10）。

模式8|温度/模型跟着任务性质走

任务性质	设置	实例
确定性（改写/拆分/分类/判断）	temperature 0-0.1	小办query_rewrite 0.1、split_and_classify 0
可控生成（答案组织/追问回答）	0.25-0.45	小办normal_answer 0.35；缓择clarify 0.4
对抗生成（唱反调/纠偏）	中低温+ 强度开关	缓择devil_advocate默认0.5 +aggressive开关（见模式10）
创意/风格化（报告/文案）	更高+风格画像约束	经历翻译官报告生成

- 原则：用流程与约束换自由度，而不是用温度赌运气。

模式9|上下文感知的动态约束（缓择特色，最容易抄的一招）

- 场景：用户最新一轮明确"不想花钱/想省钱"，唱反调agent若还从"钱"切入，等于对着用户已定的结论抬杠、甚至反向推荐更贵的选项——那是替用户做决定。
- 写法（缓择devil_advocate.py v40预算分流）：

【硬性指令 预算分流 禁止从价格切入】

用户已在最近一轮明确表态「不想花钱/想省钱/自己搞定/不报课」之类诉求。

这是用户的结论，不是你要反驳的目标。

你必须换一个与钱完全无关的角度反驳，例：

- 「你之前作息不规律，这次怎么保证能坚持下来？」
- 「这个方案需要每天 1 小时，工作忙起来那周你大概率会断」

绝对禁止任何「价格/投入产出/这钱花得值不值/应该选更全面的方案（更贵）」表述。

绝对禁止因用户省钱就推荐相反方向（更贵更全）——那是替用户做决定，不是反向建议。

- 效果：纠偏角度跟随用户实时表态切换；避免"为反对而反对"与变相推销。

模式10|强度开关代替温度：让"敢说"由Prompt控制，不由随机性赌

- 场景：缓择需要Agent 3"敢于反驳用户"，又不能真激怒用户——纯靠高温会输出不可控的冒犯话。

· 写法：输出分级字段+轮数降级+Prompt端强度开关：

```
"reverse_intensity": "strong 或 weak"
- strong：明确推荐了不同选项，或指出了具体的、可验证的硬伤
- weak：仅提示泛泛风险、未指向具体替代（降级态，只在已唱反调 2 轮且用户仍不接受时使用）
【本轮是第 {turn} 轮（最多 2 轮）】若已是第 2 轮、用户仍不接受，主动降级为 weak。
```

· 效果：产品层可以"先强后弱"控节奏，评测层可以用字段判定"这轮反了没有、强度是否合理"——把"敢不敢说"变成可配置、可回归的产品参数，而不是祈祷模型今天心情好。

模式11|征询式记忆引用：压缩历史+约束使用方式

· 写法（缓译consultative.py）：历史压缩成"画像块"（后悔模式/风格/场景记忆/偏好倾向各一行），并附使用约束：

```
[用户历史参考 - 请用征询语气使用]
历史后悔模式：冲动购买（3次）、高估自己坚持能力
决策风格：求稳、怕麻烦
场景记忆：- 租房：上次关注[通勤时长]、选了[离公司近但贵]、曾后悔[没认真看隔音]
使用方式：引用历史时必须用「你上次说.....这次还是吗？」的句式，不要冷冰冰地断言；
若本次需求与历史冲突，应主动点出矛盾。
```

· 效果：隐私友好（不默认用户记得自己说过）、防"拿旧信息断言新决定"、省token（详见§3.3与案例F）。

5.优化前后对比（六个真实案例）

案例A|小办：从"看分数兜底"到"看正文作答"

	优化前	优化后
判定依据	检索相关度分数（0.7/0.55/0.4）	检索片段里有没有业务正文
典型错误	相关度0.70且正文文明确"1个月内提交"，仍输出"制度未明确规定"	有效正文->必须作答；仅标题/无关才兜底
提示词手段	只写"根据检索结果回答"	"严禁相似度阈值兜底" + [别报] / [实测] 示例+严重违规声明
结果	多例"有答案却兜底"（守门人②③）	该分支堵死，118条实测整体91.5%、L1 100%

案例B|小办query_rewrite：从"帮用户补全"到"只做规范化"

	优化前	优化后
行为	改写时补全条件（"报销"->"出差报销"）	只纠错/术语对齐/句式对齐，禁止补条件改范围
风险	替用户假设->答非所问	意图保真，匹配靠向量检索补
手段	无优先级声明	"规则3意图保真（最高优先级）"+三种禁止行为

案例C|经历翻译官：从"模板化报告"到"个性化诊断"

	优化前	优化后
表现	六模块千篇一律"小标题+三点"，换谁的经历都一样	每条结论锚定具体经历、title定制、版块按需1-4张
手段	只描述模块主题	"去模板化硬约束"+置信度透明+5种风格画像注入

	优化前	优化后
结果	模板感强	岗位内容匹配度85%+、幻觉率<5%（产品内抽样口径）

案例D|缓择devil_advocate：从"泛泛的附和式提醒"到"可判定的真反向"

	优化前	优化后
行为	输出"也要注意风险"这类空话，不指向具体选项	必须推荐不同选项B或指出具体可验证硬伤，严禁空话
协议	无强制步骤	【强制反向协议】Steel-man -> 找矛盾-> 亮明反向-> 自我核验（不达标重写）
判定	无法程序判断"反了没有"	JSON字段challenged_preference/recommended_alternative/reverse_intensity/contradiction，测试可断言
手段	靠温度赌"敢不敢说"	协议+强度分级+轮数降级（最多2轮自动weak）
效果	反向成功率不可测（PRD TC-002 曾挂）	"反了没有/强度是否合理"成为可回归验收项

案例E|缓择预算分流：从"对着用户结论抬杠"到"换角度纠偏"

	优化前	优化后
场景	用户已说"不想花钱"，纠偏仍从"这钱值不值"切入，甚至反向推更贵方案	检测到预算敏感-> 注入"禁止从价格切入"硬指令，改从习惯/可持续性切入
本质	反驳用户已定的结论=变相推销	尊重用户结论，只挑战没想清楚的部分
手段	无上下文感知	动态约束块（budget_block）按用户最近表态注入

案例F|缓择征询式记忆：从"冷冰冰断言历史"到"征询式引用"

	优化前	优化后
行为	"你上次买了X，说明你更看重Y"（直接断言）	"你上次说续航比重量重要，这次还是吗？"（征询式）
手段	历史全文回放or无历史	画像压缩+使用句式约束+冲突主动点出
效果	隐私不适+错误归因	信任度提升（PRD记忆征询项）+token大降

6.我的Prompt自检清单（写完必过一遍，v2三项目版）

- 1. 角色是否唯一+有可判断的KPI（"避免后悔" > "当助手"）？
- 2. 每个输入字段都说明含义与可信度？禁止模型臆断？
- 3. 任务是否拆成有序步骤，写明不可跳步？
- 4. 模糊判断处有没有强制自检步骤（自我核验/分支判定）？
- 5. 有没有显式负面约束，而不只是正面要求？
- 6. 有没有把真实badcase当【别报】示例贴进去？
- 7. 兜底是否分层（不同"不知道"不同话术+人工出口）？
- 8. 输出Schema是否字段固定+语义注释+判定字段+禁止额外文字？
- 9. 有没有动态约束（跟随用户最新表态切换禁止项）？
- 10. 有没有克制边界（字数上限/不替用户决定/不无限追问/不推销）？

11. 上下文是否做了预算与压缩（固定进System、动态置User尾部、历史摘要化）？
12. 引用用户历史是否征询式（不默认、不冷断言）？
13. System是否规则编号化、重点约束放首尾？
14. 温度是否匹配任务性质；"敢说/强对抗"是否用强度开关而非温度赌？
15. 有没有可回归的验收口径（什么输出算违规/做没做到可程序判定）？

7.怎么对外讲/核心要点

核心要点

我把Prompt当"产品需求文档"写：角色设定->任务拆解->约束->Few-shot ->输出Schema五段式。

三个项目横跨事实问答、生成诊断、对抗纠偏三类任务，验证的是同一件事——把失败模式写进提示词当反例。

比如小办"库里有答案却因相似度分不够而兜底"，我直接贴错误case + "严禁相似度阈值兜底"；

缓择的魔鬼代言人连RLHF讨好倾向都要对抗，靠的是把"Steel-man ->找矛盾->亮反向->自我核验"写死成强制协议，

再配reverse_intensity强度字段让"敢不敢反驳"变成可配置可回归的产品参数，而不是用温度赌运气。

问：为什么不用few-shot堆例子？

少量高质量反例（[别报] 错误vs [实测] 正确）比一堆正例有效——正例教模仿、反例教边界；我的反例全部来自真实回归badcase。

复杂协议（如反向纠偏）会用"用户话->完整JSON"成对few-shot教结构（缓择4条）。

问：怎么验证Prompt改对了？

靠评测集回归。小办118条分层、缓择四类场景+接口校验、经历翻译官抽样核对——改一处、跑一遍、看分层指标，防止按下葫芦浮起瓢（详见《评测体系一页纸》）。

问：CoT不怕模型绕吗？

我不开放自由推理，而是把推理写成显式协议：每步一个判定点+强制自检+设兜底出口。

缓择agent3的"自我核验（不达标必须重写）"是防绕性价比最高的手段。

问：上下文超长怎么办？

先做窗口预算：固定规则进System、动态内容进User尾部、历史摘要化压缩成画像块。

缓择把历史压成"画像块+征询式引用句式"，既省token又隐私友好。

问：Prompt和RAG谁更重要？

是"约束链"的两端：RAG决定能不能找到，Prompt决定找到/没找到时怎么表现。

小办三个守门人里两个是检索侧、一个是提示词侧，必须一起改才有效。